Lecture 10 — Handlers, Services, Repositories, Middleware & Request Context

Backend Engineering from First Principles

These notes follow the supplied lecture transcript. They explain the request lifecycle inside a backend server and how the main architectural components cooperate.

1. The Big Picture: Request Lifecycle Inside a Server

A client communicates with a backend server over HTTP. Once the operating system forwards an incoming request to the port on which the server is listening, the request enters the server's internal lifecycle.

A typical flow is:

```
Client
  |
  | HTTP request
  v
Server entry point
  |
  v
Middleware (zero or more)
  |
  v
Routing / route matching
  |
  v
Middleware (zero or more)
  |
  v
Handler / Controller
  |-- Extract request data
  |-- Deserialize / bind
  |-- Validate and transform
  |
  v
Service layer
```

```
  |-- Business logic
  |-- Orchestration
  |-- Calls repositories or external services
  |
  v
Repository layer
  |-- Build database query
  |-- Execute database operation
  |-- Return database result
  |
  v
Service layer
  |
  v
Handler / Controller
  |-- Choose HTTP status
  |-- Format response
  v
HTTP response
  |
  v
Client
```

The exact implementation differs between languages and frameworks. The separation into handlers, services, and repositories is a design pattern—not a mandatory rule imposed by HTTP or by a programming language.

Why separate responsibilities?

The separation makes a codebase:

• Easier to maintain and debug.

• Easier to extend with new features.

• More scalable as the number of API endpoints grows.

• Less repetitive because each layer has a focused responsibility.

A small application can technically put everything in one handler. The layered approach becomes valuable as complexity increases.

2. Handler / Controller Layer

A **handler** (often called a **controller**) is the function associated with a matched route. It is the entry point for endpoint-specific request processing.

A typical handler receives:

• A **request object**, containing information about the incoming HTTP request.

• A **response object**, used to construct or send the HTTP response.

The framework or runtime commonly supplies these objects to the handler.

Main responsibilities

1. Understand which endpoint is being called. Routing has already matched the incoming request to this handler.

2. Extract input data from the request object.

3. Deserialize or bind the data into a format the application language can use, when this has not already been done upstream.

4. Validate and transform the input.

5. Call the service layer with clean, prepared data and relevant trusted request metadata.

6. Build the HTTP response, including the status code and response body.

Extracting request data

The input source depends on the HTTP method and API design:

Request information	Common source
Filtering, pagination, sorting	Query parameters
Resource identifier in a route	Path parameters
Data to create or update a resource	Request body
Authentication credentials and metadata	Headers, cookies, or other configured locations

The transcript gives GET query parameters and POST/PUT/PATCH request bodies as examples. DELETE requests may also carry a body depending on the API design and client/server conventions.

Binding / deserialization

The client commonly sends JSON. The backend must make that data usable in its native language representation.

Examples:

• Go: decode JSON into a struct.

• Python: decode JSON into a dictionary or a class/model.

• Rust: deserialize JSON into a struct.

• Node.js/Express: JSON parsing middleware often performs this step before the handler runs.

This conversion is often called **binding** by web frameworks.

If the request body cannot be parsed or bound correctly, the server can stop processing and return **400 Bad Request**, because the request payload is malformed or unusable.

Validation and transformation

After binding, validate the data and then apply any intended transformations.

• **Validation** checks that the supplied values satisfy the API's requirements.

• **Transformation** changes valid input into a convenient, predictable representation for downstream code.

For example, a books endpoint may accept an optional `sort` query parameter with values `name` or `date`. If the client omits it, the transformation step can supply a default such as `date`.

This means the service layer does not need repeated branches to determine what to do when the client omitted the parameter.

The transcript recommends making query parameters optional where practical, and defining sensible defaults. This is an API design guideline, not an absolute rule for every endpoint.

Keep HTTP concerns in the handler

The handler is responsible for HTTP-specific concerns:

• Reading request data.

• Interacting with request/response objects.

• Choosing an appropriate HTTP status code.

• Formatting the response sent to the client.

The service should not need to know whether its result will be returned with HTTP 200, 201, 204, or another status.

3. Service Layer

A **service** is a function or group of functions that performs the application's actual processing or business logic.

A useful design test from the lecture:

> *Looking at a service method alone should not make it obvious that it is being called by an HTTP API.*

Responsibilities

• Execute business logic.

• Coordinate the steps required to complete a use case.

• Call one or more repository methods when database access is needed.

• Call external services or APIs when required.

• Send emails or notifications when required.

• Combine or reshape results from multiple sources.

• Return data or an outcome to the handler.

Services do not have to call repositories

Not every service requires database access.

Example: a service whose only job is to send an email may accept an email address and message details, call an email provider, and return a success/failure result. No repository call is necessary.

Service as an orchestrator

A service can call multiple repository methods and combine their results.

For example, a service might:

1. Fetch a book from one repository method.

2. Fetch its author details using another method.

3. Combine the required fields.

4. Return one result to the handler.

This orchestration belongs in the service layer rather than making one repository method responsible for several unrelated operations.

Keep the service independent of HTTP

Ideally, a service:

• Accepts ordinary application data.

• Performs processing.

• Returns a result.

• Does not read HTTP request objects.

• Does not write HTTP response objects.

• Does not decide HTTP status codes.

• Does not repeat request validation that belongs at the API boundary.

This separation allows the same service logic to be reused from different entry points, such as an HTTP API, a background job, or a command-line process.

4. Repository Layer

A **repository** is the layer responsible for database operations. It receives the information needed for a database operation, constructs and executes the database query, and returns the database result.

Responsibilities

• Build database queries.

• Apply database filters, sorting, and other query conditions.

• Insert, fetch, update, or delete records.

• Return the result of the database operation.

The repository should focus on persistence rather than coordinating the entire business use case.

One method, one clear responsibility

The lecture recommends that a repository method should have a predictable return type and one clear job.

Prefer separate methods such as:

• `getAllBooks(sort)`

• `getBookById(id)`

Avoid one method whose optional argument changes whether it returns a single book or a list of books. Separate methods make their behavior easier to understand and use.

Service vs repository

Service layer	Repository layer
Implements business/use-case logic	Implements database access
Orchestrates multiple operations	Performs a focused persistence operation
May call multiple repositories	Executes a database query or write
May call external APIs, send emails, notifications	Should not own unrelated business workflows
Returns an application-level result	Returns database result/data

5. End-to-End Example: Fetching Books

Suppose the client calls:

```
GET /books?sort=name
```

A typical execution is:

1. The server receives the HTTP request.

2. Routing matches `/books` to the appropriate handler.

3. The handler extracts the `sort` query parameter.

4. The handler validates that `sort` is an allowed value.

5. If `sort` is omitted, the handler's transformation step supplies a default.

6. The handler calls a service method with the prepared data.

7. The service calls the repository method for fetching books.

8. The repository builds and executes the database query using the sorting requirement.

9. The repository returns the database result to the service.

10. The service returns the result to the handler.

11. The handler chooses the HTTP status and formats the response.

12. The server sends the response to the client.

For a successful fetch, the transcript's example returns an array of books with **200 OK**.

Response status examples mentioned in the lecture

Situation	Status
Successful fetch	200 OK
Resource created	201 Created
Successful operation with no response body	204 No Content
Invalid request / client-side failure	400 Bad Request
Server-side failure	500 Internal Server Error

The exact status depends on the endpoint's semantics and the actual outcome.

6. Middleware

A **middleware** is a function that runs at a point in the request/response lifecycle, usually before a handler or while the response is being processed.

Middleware is optional. A request can pass through several middleware functions, or go directly to routing/handler logic where the framework allows it.

Typical middleware arguments

In frameworks that use the familiar middleware pattern, a middleware receives:

1. Request — inspect or read incoming request information.

2. Response — modify response headers or send a response.

3. Next — a function that passes execution to the next processing step.

The exact function signature differs across frameworks and languages.

What does `next()` do?

`next()` passes control to the next middleware or next processing context in the framework's chain.

A middleware can:

• Perform its work and call `next()` to continue.

• Modify request/response information and then continue.

• Send a response itself and stop the chain, for example when authentication fails or a rate limit is exceeded.

Conceptual flow:

```
Request
  |
  v
Middleware A
  |  next()
  v
Middleware B
  |  next()
  v
Route matching / Handler
  |
  v
Response
```

Why use middleware?

A backend may have hundreds of endpoints and receive a very large number of requests. Many endpoints need the same cross-cutting operations.

Without middleware, common logic would be repeated in many handlers. Middleware centralizes reusable request/response processing and reduces duplication.

Common concerns include:

• Security checks and headers.

• CORS handling.

• Authentication.

• Rate limiting.

• Logging and monitoring.

• Global error handling.

• Compression.

• Serialization/deserialization or request binding, depending on the framework.

• Validation/transformation, where the application's design places them.

Middleware order matters

Middleware runs in a configured order. That order affects:

• Which checks happen first.

• Whether a request is rejected early.

• Which metadata is available to later middleware and handlers.

• Whether errors can be caught by an error-handling middleware.

A typical illustrative sequence from the lecture is:

1. CORS handling.

2. Logging.

3. Authentication.

4. Route/handler processing.

5. Global error handling in the framework's appropriate error path.

This is an example, not a universal ordering rule. Frameworks differ in how they handle errors and response unwinding.

7. Common Middleware Examples

7.1 CORS middleware

CORS (Cross-Origin Resource Sharing) is a browser-enforced mechanism that controls whether a web page from one origin can access a resource from another origin.

The server examines the request's `Origin` and, when appropriate, returns CORS response headers that tell the browser which origins are allowed.

Why middleware?

• The policy may need to be applied to many or all API endpoints.

• It reads request information and modifies response headers.

• Centralizing it avoids repeating the logic in every handler.

Important: CORS is not authentication. It is a browser access-control mechanism; non-browser clients are not restricted by browser CORS enforcement.

7.2 Security headers middleware

This middleware adds response headers that help browsers apply security policies. The lecture mentions Content Security Policy as an example.

It is commonly centralized because the same security headers may need to be applied across many responses.

7.3 Authentication middleware

Authentication middleware typically:

1. Extracts a credential or token from the configured request location, such as a header or cookie.

2. Verifies the credential.

3. If verification fails, sends an authentication error (commonly **401 Unauthorized**) and stops processing.

4. If verification succeeds, obtains trusted identity/metadata such as user ID, role, or permissions.

5. Stores the relevant metadata in the request context.

6. Calls `next()` so downstream code can continue.

The lecture emphasizes that identity information should come from verified authentication context—not from an untrusted user ID supplied in the request body.

7.4 Rate-limiting middleware

Rate limiting restricts how many requests a client can make within a chosen time window.

A policy might track requests by IP address or another client identity. The threshold and time window are application-specific.

• If the client exceeds the limit, the server can return **429 Too Many Requests**.

• Otherwise, middleware calls `next()`.

The lecture's numbers are illustrative only; they are not recommended universal limits.

7.5 Logging and monitoring middleware

Logging middleware can record details such as:

• HTTP method.

• Request path.

• Query parameters.

• Relevant request metadata.

• Request ID.

• Outcome and timing, if instrumented.

Logs help with debugging, auditing, monitoring, and understanding application behavior. Avoid logging secrets, passwords, access tokens, or unnecessary sensitive payload data.

7.6 Global error-handling middleware

Errors can occur in middleware, handlers, services, or other parts of the application.

A global error handler provides a centralized place to:

• Receive errors through the framework's error-handling mechanism.

• Distinguish expected client errors from unexpected server errors.

• Choose an appropriate status code.

• Return a consistent, structured error response.

The lecture describes placing error handling at the end of the middleware chain. The exact placement and signature are framework-specific. In frameworks with a dedicated error-middleware mechanism, errors must be forwarded in the way that framework expects; ordinary `next()` behavior may not be sufficient.

A structured error response may include:

• A human-readable message.

• A stable application error code or ID.

• An HTTP status.

Do not expose internal stack traces or secrets to clients in production.

7.7 Compression middleware

Compression middleware can compress large responses before they are sent over the network. The client/browser decompresses the response when supported and configured.

This can reduce response size and network transfer, with some CPU cost.

8. Request Context

A **request context** is request-scoped storage or state associated with one particular HTTP request.

Each request has its own context. It is not global state shared by every client.

Why do we need it?

A request may pass through many middleware functions and handlers. One component may discover information that later components need.

The context allows components in the same request lifecycle to share metadata without passing every value manually through every function call.

It is commonly implemented as key-value data, although concrete implementations vary by language and framework.

Typical lifecycle

```
Incoming request
   |
   v
Authentication middleware
   |-- Verify token
   |-- Determine user ID / role / permissions
   |-- Store trusted metadata in request context
   |
   v
Later middleware / Handler
   |-- Read metadata from the same request context
   |-- Perform authorization or business operation
   |
   v
Response
```

Example 1: Store authenticated user identity

Suppose a user sends a request to create a book.

A secure flow is:

1. Authentication middleware verifies the user's token.

2. It determines the authenticated user's ID and role.

3. It stores those trusted values in the request context.

4. The book handler reads the authenticated user ID from context.

5. The handler/service uses that identity when creating the book.

Do **not** trust a user ID merely because the client included it in the request body. A malicious client could submit another person's ID. Use identity established by the authentication process.

Example 2: Request ID / correlation ID

A middleware can generate a unique request ID (for example, a UUID) and store it in the request context.

The application can then:

• Include it in logs.

• Pass it to downstream services, often in a request header.

• Use it to trace one request across multiple services.

• Correlate events when debugging or auditing.

Example 3: Cancellation, deadlines, and abort signals

Request context mechanisms may also carry cancellation signals and deadlines to downstream operations. This helps prevent work from continuing indefinitely after the original request is cancelled or its deadline has expired.

The exact APIs for cancellation and deadlines depend on the language/framework.

Request context vs global state

Request context	Global state
Scoped to one request	Shared more broadly by the application/process
Holds metadata for that request	May be visible to unrelated requests
Helps prevent cross-request data mixing	Can cause unsafe data sharing if used incorrectly
Examples: user ID, role, request ID, deadline	Examples: application configuration, shared service instances

Never store one user's request-specific identity in a mutable global variable. Concurrent requests could overwrite one another's data.

9. How the Concepts Fit Together

Component	Main question it answers	Primary responsibility
Routing	“Which endpoint is this?”	Match the request to a handler
Handler / Controller	“How does this HTTP request enter and leave the application?”	Extract, bind, validate/transform, call service, create HTTP response
Service	“What does the application need to do?”	Business logic and orchestration
Repository	“How do we access persisted data?”	Database queries and writes

Component	Main question it answers	Primary responsibility
Middleware	“What common work should happen around many requests?”	Reusable cross-cutting request/response processing
Request context	“What metadata belongs to this one request?”	Share request-scoped state across middleware and handlers

A useful mental model

• **Handler:** HTTP boundary and traffic coordinator.

• **Service:** business/use-case brain.

• **Repository:** database access specialist.

• **Middleware:** reusable checkpoint in the request/response pipeline.

• **Request context:** shared notepad for one request.

10. Practical Design Checklist

When implementing an endpoint, ask:

• [ ] Does routing map the URL and HTTP method to the correct handler?

• [ ] Does the handler extract the correct path, query, header, and body values?

• [ ] Is request data parsed/bound into the application's native representation?

• [ ] Is all untrusted input validated at the API boundary?

• [ ] Are optional inputs given sensible defaults where appropriate?

• [ ] Are transformations explicit and predictable?

• [ ] Does the handler pass clean data and trusted metadata to the service?

• [ ] Is business logic kept out of the handler?

• [ ] Does the service orchestrate repository/external operations?

• [ ] Does each repository method have one clear responsibility?

• [ ] Are HTTP status codes and response formatting handled at the HTTP boundary?

• [ ] Are common concerns implemented once in suitable middleware?

• [ ] Is middleware order intentional?

• [ ] Is authenticated identity obtained from trusted context rather than client-supplied identity?

• [ ] Is request-specific data isolated to the request context?

• [ ] Are errors returned consistently without exposing internal details?

11. Interview Questions and Answers

Q1. What is the difference between a handler and a service?

A handler deals with HTTP: it extracts request data, validates/binds it, calls the service, and builds the response. A service contains business logic and should not depend on HTTP request/response objects.

Q2. What is the repository pattern?

It separates database access from business logic. Repository methods build and execute database operations and return the results to the service layer.

Q3. Why should a service not directly use the HTTP response object?

It couples business logic to HTTP. Keeping the service independent makes it reusable from other entry points and easier to test.

Q4. Can a service work without a repository?

Yes. A service may send an email, call an external API, or perform processing that does not require database access.

Q5. Can one service call multiple repositories?

Yes. The service layer orchestrates multiple repository operations and can combine their results.

Q6. What does `next()` do in middleware?

It passes control to the next processing step in the framework's chain. A middleware may instead send a response and stop the chain.

Q7. Why is middleware useful?

It centralizes common logic—such as authentication, logging, CORS, rate limiting, and security headers—so that the same code is not duplicated across handlers.

Q8. Why does middleware order matter?

Because each middleware receives the request after earlier processing. The order determines when checks happen, what metadata is available, and whether errors can be handled correctly.

Q9. What is request context?

It is state scoped to one request and shared with the middleware/handlers participating in that request lifecycle.

Q10. Why should user ID come from authenticated context instead of the request body?

The body is controlled by the client and can be manipulated. The authenticated identity is derived from verified credentials and is therefore the trusted source for the caller's identity.

Q11. What is the difference between authentication middleware and authorization?

Authentication establishes who the caller is. Authorization checks whether that authenticated caller is allowed to perform a particular action. The lecture discusses roles/permissions being made available to downstream checks.

Q12. What status code is appropriate when a rate limit is exceeded?

429 Too Many Requests.

Q13. What is the purpose of global error handling?

It provides a consistent way to convert application errors into structured HTTP error responses with appropriate status codes.

Q14. What is a request ID useful for?

It helps correlate logs and trace one request across multiple services or components.

12. Quick Revision Sheet

• **Handler/controller:** HTTP-facing layer; extracts input, binds/parses, validates/transforms, calls service, sends response.

• **Service:** business logic and orchestration; may call repositories, external APIs, or notification services.

• **Repository:** focused database access; builds and executes queries/writes.

• **Middleware:** reusable function in the request/response chain; commonly receives request, response, and `next`.

• `next()`**:** passes execution to the next processing context.

• **Request context:** request-scoped key-value state shared across relevant middleware and handlers.

• **CORS:** browser-enforced cross-origin access mechanism; not authentication.

• **Authentication middleware:** verifies credentials and stores trusted identity metadata in context.

• **Rate limiting:** restricts request frequency; common response is 429.

• **Global error handling:** standardizes error responses.

• **Request ID:** correlates logs and traces a request across services.

• **Core principle:** keep HTTP concerns, business logic, and database access separated.

Remember the flow

```
Request
  -> Middleware
  -> Routing
  -> Handler
  -> Service
  -> Repository
  -> Service
  -> Handler
  -> Response
```

Middleware may appear at multiple points, and the exact chain depends on the framework.